

Information Processing and Retrieval

UA.DETI.PRI - 2025/26

Tiago Almeida

How to create a project in python

Industry standard

Why do we care?

- ❖ Isn't it good enough to just put my code in a folder and write a requirement.txt file?
 - For a tiny, one-off script you'll never use again, this maybe good enough.
- ❖ But as you need to share it it will start to break apart.
 - Code is hard to read (no code standard)
 - What python version?
 - “Hidden” dependencies will break your code



The Modern Python project

- ❖ pyproject.toml is the current industry standard to orchestrate a python project.
 - It was introduced in PEP 518 and extended in PEP 621
- ❖ Core responsibilities:
 - It defines project metadata
 - Lists core application dependencies
 - Specifies the build system
 - Allows for definition of development tools like:
 - pyright
 - ruff
 - pytest
 - etc..

```
[project]
name = "Sapien"
version = "0.1.0"
description = "Template for classes - RI 2025"
authors = [
    {name = "Tiago Almeida", email = "tiago.almeida@ua.pt"}
]
readme = "README.md"
requires-python = ">=3.13"
dependencies = [
    "pyarrow>=19.0.1",
    "pydantic>=2.11.9",
    "pydantic-core>=2.33.2",
    "fastapi[standard]>=0.116.1",
    "uvicorn>=0.35.0",
    "psutil>=7.0.0",
]

[dependency-groups]
dev = [
    "mypy>=1.8.0",
    "ruff>=0.1.0",
    "pyright>=1.1.0",
    "pre-commit>=3.6.0",
]

[project.scripts]
cli = "sapien.entrypoints.cli:main"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Project manager tool

- ❖ pyproject.toml is just a standard on how to define a project, now we need a tool to actually build our project.
- ❖ Some example
 - uv
 - poetry
 - Hatch
- ❖ We are going to use uv
 - Written in rust
 - Extremely fast and easy to use

uv sync and uv run

How to define code standards?

- ❖ One of the big advantages of `pyproject.toml` is that allows us to also define other tools, like type-checking, linting and testing.
 - We mostly care about type-checking and linting.
- ❖ For type-checking we are going to use *pyright*
- ❖ For linting we are going to use *ruff*

Type-checking in python

- ❖ Type checking is the process of verifying the type safety of a program.
 - Acts like a rigorous grammar and logic checker for your code.
 - It finds errors before you run the program!
- ❖ You specify this as type hints (type annotation)
 - Introduced in PEP 484

```
def reader(file_path: str) -> Iterator[dict[str, str]]:  
    """Reader for the IPC file."""
```

```
[tool.pyright]  
include = ["src"]  
exclude = ["**/__pycache__"]  
typeCheckingMode = "strict"  
reportImportCycles = false  
reportUnnecessaryCast = true  
reportUnnecessaryTypeIgnoreComment = true  
pythonVersion = "3.13"  
pythonPlatform = "All"
```

- ❖ Pyright is the tool that **statically analyzes** your code to **enforce** these type hints.

Linting in Python

- ❖ Linting is the process of automatically checking your code for stylistic and programmatic errors.
 - It acts like a **style guide** for your code.
 - It is capable to automatically format your code to follow code standards.
- ❖ *ruff* is a tool that formats your code according to predefined rules.

```
[tool.ruff]
line-length = 100
src = ["src"]
```

```
[tool.ruff.lint]
ignore = [
    "COM812", # -- over-aggressively insists on trailing commas
]
select = [
    "C4",      # -- flake8-comprehensions --
    "COM",     # -- flake8-commas --
    "D200",    # -- short docstring on one line --
    "D205",    # -- docstring: blank line between summary line and
    "D209",    # -- multi-line docstring closing quotes on own
    "D212",    # -- docstring summary line starts immediately after
    "E",       # -- pycodestyle errors --
    "F",       # -- pyflakes --
    "I",       # -- isort (imports) --
    "PLR0402", # -- Name compared with itself like `foo == foo`
    "PT",      # -- flake8-pytest-style --
    "SIM",     # -- flake8-simplify --
    "UP006",   # -- use built-in generics like `list` instead of
    "UP007",   # -- prefer `A | B` over `Union[A, B]` --
    "UP008",   # -- use modern `super()` call without arguments
    "UP015",   # -- redundant `open()` mode parameter (like "r"
    "UP017",   # -- use `datetime.UTC` instead of `datetime.time
    "UP018",   # -- unnecessary `{literal_type}` call like `str("
    "UP032",   # -- use f-string instead of `.format()` call --
    "UP034",   # -- avoid extraneous parentheses --
    "UP040",   # -- prefer `type Xyz = ...` over `Xyz = TypeAli
    "W291",    # -- remove trailing whitespace from end of line
```


How to create a project in python